

FACULDADE ENGENHEIRO SALVADOR ARENA

Guilherme Alves Barbosa - 082220014

Lucas Junqueira Gonçalves – 082230029

Murilo Umbelino dos Santos – 082230013

Victor Mendes de Andrade Ferreira - 082230015

Comunicação de Dados

Simulador OSI

Vinícius Borges

09/2026

São Bernardo do Campo

1. Visão geral

O programa desenvolvido em Python simula uma rede de computadores e roteadores e demonstra o percurso de uma mensagem de um computador ao outro, passando pelos roteadores quando estão presentes no caminho. Os acontecimentos são exibidos através de uma sequência determinística de eventos e a visualização da mensagem sendo repassada por cada camada do modelo OSI é exibida em tela. Cada computador contém sete camadas e cada roteador contém somente três: física, enlace e rede. A comunicação desce L7 até L1 na origem, sobe até L3 e desce novamente até L1 em cada roteador até, enfim, subir de L1 a L7 no destino.

2. Separação entre as camadas

A arquitetura do código reflete o isolamento das camadas do modelo OSI através de classes independentes, cada uma com sua responsabilidade bem definida. O fluxo de dados é gerenciado pelos métodos *descendo(...)* (encapsulamento na origem) e *subindo(...)* (desencapsulamento no destino/roteadores). O motor principal coordena essas passagens, transferindo a PDU (Protocol Data Unit) modificada de uma camada para a adjacente.

A interface entre as camadas é definida pelos argumentos recebidos e pelo resultado devolvido por seus métodos. As operações retornam um objeto *ResultadoCamada*, que contém valor, ação e descrição. O campo valor transporta o resultado do processamento e os demais descrevem a operação para o registro de eventos. O simulador recebe esse resultado e o fornece à etapa adjacente do percurso. Na recepção, por exemplo, a camada física entrega bytes à camada de enlace. Após validar o quadro, a camada de enlace entrega à camada de rede somente os bytes do pacote. Se a validação falha, retorna valor=None, e o motor interrompe o processamento naquele receptor antes de chamar a camada de rede.

Camada	Entrada na descida / saída na subida	Responsabilidade
L7 Aplicação	Texto e nomes de processo / texto entregue	Produz e entrega a mensagem e não codifica UTF-8
L6 Apresentação	Texto / texto	Codifica UTF-8 e aplica XOR 0x5A e decifra somente no destino
L5 Sessão	Bytes de apresentação / bytes sem H5	Abre, mantém e encerra o diálogo e insere ou remove o identificador de 4 B
L4 Transporte	Bytes de sessão / bytes remontados	Insere portas e sequência, segmenta, demultiplexa e ordena

Camada	Entrada na descida / saída na subida	Responsabilidade
L3 Rede	Bytes de segmento / bytes sem H3 no destino	Insere IPs e identificador e consulta a tabela e escolhe o próximo salto
L2 Enlace	Bytes de pacote / bytes sem H2/F2	Cria o quadro para o vizinho, verifica MAC e CRC32
L1 Física	Bytes de quadro / bytes recebidos	Converte cada octeto em oito bits e reconstrói os octetos

Cada camada interpreta os campos correspondentes à sua responsabilidade. A camada de enlace, por exemplo, verifica o endereço MAC de destino e o CRC do quadro. Quando a verificação é bem-sucedida, entrega o conteúdo do quadro à camada de rede. Se houver erro, descarta o quadro e o motor interrompe o processamento naquele receptor, sem acionar a camada superior.

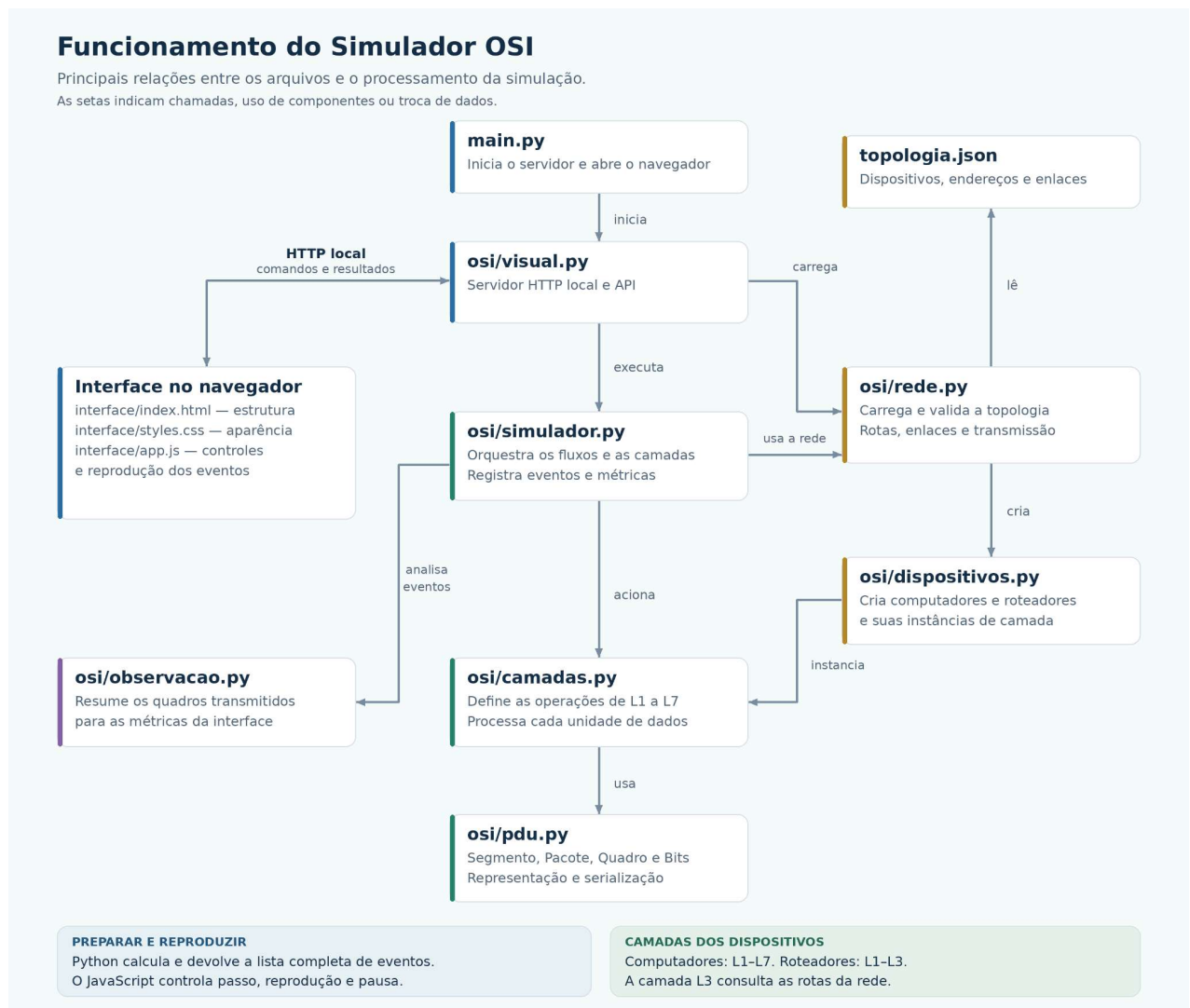
A separação também aparece na representação dos dispositivos e das unidades de dados. Os computadores possuem objetos para as sete camadas, enquanto os roteadores possuem somente as camadas física, enlace e rede. O objeto Pacote, utilizado pela camada de rede, contém os endereços IP, o identificador e a carga em bytes. Ele não possui atributos de porta nem uma referência ao objeto Segmento originalmente criado pelo transporte.

Por isso, o roteador do simulador não dispõe de uma operação de transporte para consultar as portas. Ao receber um pacote, interpreta o cabeçalho de rede e utiliza o endereço IP de destino para escolher o próximo salto. A carga permanece como bytes não interpretados e segue para um novo enquadramento na camada de enlace. No computador de destino, o segmento é entregue à L4, que interpreta as portas e realiza a demultiplexação e a remontagem.

Os bytes das portas continuam presentes na carga do pacote. Portanto, essa separação representa o comportamento definido pela arquitetura do simulador, e não uma impossibilidade de examinar esses bytes.

A separação pode ser conferida pela existência de somente L1, L2 e L3 nos roteadores, pela ausência de atributos de porta em `Pacote` e pelo encerramento do processamento no receptor quando a camada de enlace detecta um CRC inválido. No cenário E6, o descarte ocorre em L2 de R3, sem processamento posterior em L3 desse roteador.

3. Diagrama dos módulos



O arquivo `main.py` cria uma instância de `InterfaceLocal`, definida em `osi/visual.py`, que inicia o servidor HTTP local e abre o navegador. A interface utiliza `index.html` para a estrutura da página, `styles.css` para a apresentação e `app.js` para enviar comandos e exibir os resultados. O módulo `visual.py` carrega a rede por meio de `Topologia`, definida em `rede.py`, que lê `topologia.json` e cria os computadores, roteadores e interfaces de rede. Cada dispositivo instancia suas camadas, definidas em `camadas.py`. O simulador coordena o processamento, utiliza as representações binárias de `pdu.py` e recorre a `observacao.py` para auxiliar na análise dos eventos.

Ao acionar Preparar, o núcleo Python calcula a simulação e devolve a lista completa de eventos. Os controles Próximo passo, Executar e Pausar reproduzem essa lista no navegador. A velocidade altera somente o intervalo de apresentação, sem modificar as rotas ou a quantidade de bytes transmitidos.

4. Estrutura da unidade de dados

Componente	Entrada	Saída / efeito
Topologia	Caminho JSON ou objeto de dados	Dispositivos, interfaces, segmentos e tabelas validados
Simulador.configuracao	Código E1-E7, C1-C7 ou PERSONALIZADO	Parâmetros do cenário, com defaults
Simulador.simular	Código e alterações opcionais	ResultadoSimulacao com eventos, métricas e mensagens entregues
CamadaRede.encaminhar	Pacote, dispositivo, topologia, enlaces inativos	Próximo salto ou descarte explícito. A decisão ocorre em L3
CamadaTransporte.subindo	Segmento em bytes e contexto de IPs vindo de L3	Segmento interpretado, chave do fluxo e mensagem completa ou None
InterfaceLocal	Caminho de topologia e pasta de interface	Servidor local. Preserva a janela diante de erro de entrada
ResultadoSimulacao.to_dict	Resultado concluído do núcleo	Resposta serializável para a interface, incluindo linhas de log
detalhar_cabecalhos	Bytes da PDU e descritores dos blocos	Campos, posições e tamanhos dos cabeçalhos presentes e leitura exclusiva do observador
resumir_quadros	Eventos da simulação	Dados de aplicação, controle e eficiência de cada quadro efetivamente transmitido

A unidade de dados assume diferentes representações ao percorrer as camadas. No código, as classes Segmento, Pacote, Quadro e Bits representam essas estruturas. A tabela apresenta seus campos, os tipos utilizados no Python e a função de cada um.

Estrutura	Campo	Tipo Python	Função
Segmento	porta_origem	int	Identifica a porta do processo emissor.
Segmento	porta_destino	int	Identifica a porta do processo destinatário.
Segmento	sequencia	int	Indica a posição do segmento na mensagem, permitindo ordenar as partes recebidas.
Segmento	total	int	Informa quantos segmentos são necessários para completar a mensagem.
Segmento	payload	bytes	Transporta uma parte dos dados recebidos da camada de sessão.
Pacote	ip_origem	str	Identifica o endereço IPv4 do computador emissor.
Pacote	ip_destino	str	Identifica o endereço IPv4 de destino, utilizado na decisão de encaminhamento.
Pacote	identificador	int	Identifica o pacote, mantendo sua identificação ao longo do percurso.
Pacote	payload	bytes	Transporta o segmento serializado, tratado como carga não interpretada pela camada de rede.
Quadro	mac_origem	str	Identifica a interface que transmite o quadro no salto corrente.
Quadro	mac_destino	str	Identifica a interface que recebe o quadro no salto corrente.
Quadro	payload	bytes	Transporta o pacote serializado pelo enlace.
Bits	sequencia	str	Representa os bits transmitidos como uma sequência de caracteres 0 e 1.

Os tipos apresentados correspondem à representação em memória. Na serialização, os campos são convertidos para os formatos binários definidos pelo simulador. Os endereços IPv4, por exemplo, são armazenados como texto nos objetos e ocupam quatro octetos cada no cabeçalho de rede.

A mensagem da aplicação é representada inicialmente como texto. A camada de apresentação a converte em bytes e aplica a cifra. A camada de sessão acrescenta um identificador de quatro octetos diretamente a esses bytes, antes da segmentação.

O CRC32 é calculado durante a serialização do quadro e acrescentado como finalizador. Na recepção, seu valor é verificado antes de liberar o conteúdo para a camada de rede.

Os registros de Evento armazenam informações para apresentar e acompanhar a simulação. Esses metadados são separados das unidades transmitidas: informações disponíveis para a interface não se tornam, por isso, campos acessíveis aos roteadores.

5. Interface dos módulos

Os módulos se comunicam por chamadas de funções e métodos, com entradas e resultados definidos. A tabela apresenta os principais componentes e seus contratos de utilização.

Função ou classe	O que recebe	O que devolve	O que faz
main() — main.py	Não recebe argumentos.	Código de saída 0 ao encerrar normalmente.	Localiza a topologia, cria a interface local, abre o navegador e mantém o servidor em execução.
Topologia — rede.py	Caminho do arquivo JSON ou um dicionário com os dados da rede.	Uma instância de Topologia.	Valida a configuração e cria os dispositivos, interfaces e enlaces utilizados na simulação.
Computador e Roteador — dispositivos.py	Nome, interfaces de rede e posição no mapa.	Uma instância do dispositivo correspondente.	Organizam as interfaces e instanciam as camadas: L1–L7 nos computadores e L1–L3 nos roteadores.
Simulador.configuracao() — simulador.py	Identificação do cenário.	Dicionário com os parâmetros do cenário.	Combina os valores padrão do simulador com as configurações existentes na topologia.
Simulador.simular() — simulador.py	Identificação do cenário e alterações opcionais de configuração.	Objeto ResultadoSimulacao.	Valida os parâmetros, coordena o processamento dos fluxos e reúne eventos, métricas, rotas e entregas.
Métodos descendo() e subindo() das camadas — camadas.py	Dados a processar e parâmetros específicos de cada camada.	Objeto ResultadoCamada.	Executam as operações de envio e recepção, respeitando a responsabilidade de cada camada.
CamadaRede.encaminhar() — camadas.py	Pacote, topologia, identificação do dispositivo e enlaces inativos.	ResultadoCamada com a decisão de encaminhamento ou com valor=None.	Consulta a rota para o IP de destino e determina o próximo salto ou o descarte por ausência de rota.

CamadaTransporte.subind o() — camadas.py	Segmento serializado em bytes e endereços IP de origem e destino.	ResultadoCamada cujo valor contém o segmento interpretado, a chave do fluxo e a mensagem remontada, quando completa.	Interpreta H4, separa os fluxos, armazena as partes recebidas e remonta a mensagem na ordem correta.
to_bytes() de Segmento, Pacote e Quadro — pdu.py	Os campos armazenados na própria instância.	Sequência de bytes.	Serializa a unidade de dados no formato binário correspondente. No quadro, também calcula e acrescenta o CRC32.
from_bytes() de Segmento, Pacote e Quadro — pdu.py	Sequência de bytes da unidade correspondente.	Uma instância de Segmento, Pacote ou Quadro.	Interpreta os campos e valida o formato recebido. Na leitura do quadro, verifica o CRC32. Dados inválidos provocam uma exceção.
ResultadoSimulacao.to_dic t() — simulador.py	Os dados armazenados no resultado da simulação.	Dicionário serializável em JSON.	Organiza os eventos, as métricas e os demais resultados para envio à interface, incluindo o registro textual.
InterfaceLocal — visual.py	Caminho da topologia e, opcionalmente, caminho dos arquivos da interface.	Uma instância de InterfaceLocal.	Carrega a topologia e configura o servidor HTTP local, seus recursos e o tratamento dos comandos da interface.
InterfaceLocal.executar() — visual.py	Não recebe argumentos adicionais.	Não devolve um resultado de simulação.	Mantém o servidor atendendo às requisições até seu encerramento.
resumir_quadros() — observacao.py	Lista de eventos da simulação.	Lista de resumos dos quadros transmitidos.	Calcula a quantidade de dados, de controle e a eficiência de cada quadro efetivamente enviado.

O objeto ResultadoCamada padroniza o retorno das operações das camadas. Seu campo valor contém o resultado do processamento, enquanto acao e descricao fornecem informações para o registro de eventos. O conteúdo de valor depende da operação realizada.

Na recepção pelo transporte, a mensagem remontada permanece como None enquanto faltam segmentos. Na recepção pelo enlace, valor=None indica descarte, e o motor interrompe o processamento naquele receptor antes de acionar a camada superior.

A interface no navegador envia comandos ao servidor local e recebe os resultados em JSON. O arquivo app.js utiliza esses resultados para desenhar a rede, apresentar as unidades de dados e reproduzir os eventos.

6. Parâmetros Configuráveis

Os parâmetros da simulação são definidos pelas configurações dos cenários e podem ser ajustados nos campos da interface. Os valores padrão apresentados correspondem à topologia de referência.

Parâmetro	Valor padrão	Faixa válida e configuração
Cenário	E2 na topologia de referência.	E1 a E7, quando definidos na topologia, e Personalizado. Selecionado no campo Cenário.
Arquivo de topologia	topologia.json, na mesma pasta do executável	Arquivo JSON válido. Pode ser substituído em disco ou carregado pelo botão Abrir topologia. Recarregar arquivo lê novamente o arquivo associado à inicialização.
Origem e destino	Definidos pelo cenário selecionado.	A origem deve ser um computador da topologia. O destino pode ser o nome de um computador ou um endereço IPv4 válido, diferente do endereço de origem.
Mensagem	42 B. E7 utiliza 100 B.	Texto com tamanho de 1 a 4096 B após codificação UTF-8. Alterado no campo Mensagem.
Portas	Origem 5210 e destino 443, com exceções por cenário.	Números inteiros de 1 a 65535, definidos nos campos de portas.
Processos	Origem navegador e destino servidorWeb, com exceções por cenário.	Texto não vazio, com até 60 caracteres, definido nos campos de processos.
Segundo fluxo de E3	Origem H2 e porta de origem 5310.	Computador existente e porta entre 1 e 65535. O segundo fluxo deve diferir do primeiro pela origem ou pela porta de origem.
Carga máxima de L4	64 B. E7 utiliza 40 B.	Número inteiro de 4 a 1400 B, configurado no campo Carga máxima L4.

Queda de enlace	Desativada. E4 desativa R1–R4.	Seleção de um enlace existente no campo Derrubar enlace. A interface permite selecionar um enlace por vez.
Erro de bit	Desativado. E6 seleciona R4–R3.	Seleção de um enlace existente no campo Alterar 1 bit no enlace. Um bit é alterado no primeiro quadro que atravessa o enlace selecionado, preservando o CRC original.
Ordem inversa	Desativada.	Ativada ou desativada pela opção Enviar segmentos em ordem inversa. Altera a ordem de envio, mantendo a numeração dos segmentos.
Velocidade de reprodução	1×	Velocidades de 0,5x, 1x, 2x e 4x
Pilha exibida	OSI.	OSI ou TCP/IP, selecionados no controle Pilha. A alteração modifica somente o agrupamento visual das camadas.

O carregamento de uma topologia pela interface altera a rede utilizada em memória, sem sobrescrever o arquivo em disco. A velocidade de reprodução modifica somente o intervalo de apresentação dos eventos, sem alterar as rotas, os tamanhos das unidades de dados ou as métricas.

Os tamanhos dos cabeçalhos e a chave da cifra são valores fixos adotados pelo modelo, sem campos para alteração na interface:

Elemento	Valor adotado	Configuração
Cabeçalho de sessão — H5	4 B	Fixo no código.
Cabeçalho de transporte — H4	8 B	Fixo no código.
Cabeçalho de rede — H3	20 B	Fixo no código.
Cabeçalho de enlace — H2	14 B	Fixo no código.
Finalizador de enlace — F2	4 B	Fixo no código e transporta o CRC32.
Cabeçalhos adicionais de L6 e L7	0 B	Não são acrescentados no modelo.
Chave da cifra XOR	0x5A	Valor adotado pela camada de apresentação, sem ajuste pela interface.

7. Funcionamento Interno

O simulador carrega a topologia da rede e utiliza os parâmetros selecionados para definir a comunicação. Ao preparar a simulação, o programa processa a mensagem pelas camadas dos dispositivos e registra cada operação como um evento. A interface reproduz essa sequência, permitindo acompanhar o percurso dos dados. Os subtópicos seguintes detalham o encapsulamento, o encaminhamento, a remontagem das mensagens e o tratamento de erros.

7.1 Encapsulamento e cada passo de E2

A mensagem padrão contém 42 B. L6 codifica e cifra sem expandir o comprimento. L5 insere 4 B. Como a carga de sessão tem 46 B e o limite deste cenário é 64 B, L4 gera um único segmento de 54 B. L3 acrescenta 20 B, formando P1 com 74 B. L2 acrescenta 14 B e 4 B, produzindo Q1 com 92 B. L1 transmite 736 bits. R1 reconstrói os octetos em L1. L2 verifica o CRC e o endereço MAC de destino. Quando a verificação é bem-sucedida, remove H2 e F2, descarta a estrutura do quadro Q1 e entrega os 74 B do pacote à camada de rede. O descarte do quadro recebido não representa perda do pacote: seu conteúdo continua sendo processado por L3. L3 escolhe a saída e1, via R4, com custo restante 2. L2 constrói Q2 com os MACs de R1/e1 e R4/e0. O procedimento se repete em R4 e R3. Cada transmissão utiliza um novo quadro: Q1 em H1–R1, Q2 em R1–R4, Q3 em R4–R3 e Q4 em R3–H4. Os endereços MAC correspondem às interfaces de cada salto, e o CRC é calculado para cada novo quadro. O pacote mantém o identificador P1 e os endereços IP de origem e destino ao longo do percurso. O destino H4 remove H3. L4 interpreta as portas, recompõe a mensagem e só então entrega a L5. L5 remove H5, L6 decifra e L7 entrega ao processo servidorWeb. Após o término, o motor registra um evento local de encerramento de sessão na origem.

7.2 Roteamento e topologia

A L3 do host entrega diretamente a outro host no mesmo prefixo ou encaminha ao gateway da LAN. A L3 do roteador consulta uma tabela derivada da topologia. As LANs têm custo de encaminhamento zero. A comparação de caminhos de mesmo custo utiliza a sequência lexicográfica completa dos nomes dos roteadores, garantindo resultado determinístico. O programa armazena o caminho efetivamente transmitido. Em E6, a rota termina em R3, onde o quadro falha e não aparece a entrega a H4. Na interface, azul representa enlaces já transmitidos e amarelo representa o salto atual. No resultado de uma mensagem segmentada, a rota mostrada é a do último segmento tentado e todos os saltos de todos os segmentos continuam registrados nos eventos. A topologia admite de 2 a 24 dispositivos, ao menos dois hosts, uma interface por host e uma ou mais por roteador. Cada LAN tem exatamente um gateway e enlaces p2p ligam dois roteadores. Enlaces paralelos entre o mesmo par não são suportados. Todas as interfaces pertencem a exatamente um segmento. IPs e MACs são únicos, prefixos LAN não se sobrepõem e os custos são inteiros e não negativos.

7.3. Segmentação, demultiplexação e E7

No cenário E7, a mensagem possui 100 B úteis. A camada de sessão acrescenta H5 de 4 B, entregando 104 B ao transporte. Com limite de carga de 40 B, L4 produz três partes de 40, 40 e 24 B. A primeira contém H5 e 36 B de dados e as demais contêm 40 e 24 B de dados. Cada parte recebe H4 de 8 B, formando segmentos completos de 48, 48 e 32 B, que são encapsulados em pacotes próprios e encaminhados pela rede.

No destino, L4 identifica o fluxo, interpreta a numeração dos segmentos e armazena suas cargas. Somente após receber todas as partes, reúne os 104 B na ordem correta e os entrega a L5. A camada de sessão remove H5, a camada de apresentação decifra os 100 B restantes e a aplicação recebe a mensagem original. Esse procedimento também permite remontar a mensagem quando os segmentos são enviados em ordem inversa.

7.4. Tratamento de quadros corrompidos

Na recepção, L1 reconstrói os octetos a partir dos bits e os entrega à camada de enlace. L2 verifica a integridade do quadro comparando o CRC32 recebido no finalizador com o CRC32 calculado sobre o cabeçalho de enlace e sua carga. Se os valores forem diferentes, o quadro é considerado corrompido.

No cenário E6, um bit do quadro é alterado durante a transmissão no enlace R4–R3, sem atualizar o CRC transportado. Ao receber esse quadro, L2 de R3 identifica a inconsistência e retorna um resultado de descarte. O motor registra a ação DESCARTA e interrompe o processamento naquele receptor antes de chamar L3. Portanto, o pacote não é encaminhado para H4, e a mensagem não chega à aplicação de destino.

O quadro corrompido permanece contabilizado no total de bytes transmitidos, pois atravessou o enlace antes de ser descartado. O simulador não realiza retransmissão automática. Ao final do fluxo interrompido, a sessão emissora é encerrada como abortada por um evento local, sem transmissão de bytes adicionais.

8. Convenções de simulação

As convenções definem as regras de modelagem e contabilização utilizadas pelo simulador. Elas determinam os tamanhos das unidades de dados, as decisões de encaminhamento e os resultados apresentados.

Item	Convenção
Cabeçalhos	H5 4 B; H4 8 B; H3 20 B; H2 14 B; F2 4 B
Sessão	H5 inserido uma única vez antes de segmentar
Segmentação	Limite é a carga máxima de L4, não o tamanho do segmento completo
Contagem útil	Comprimento UTF-8 da mensagem antes da cifra
Contagem transmitida	Soma dos quadros que efetivamente entram em cada enlace, incluindo o quadro corrompido
Numeração	P1.. por segmento e Q1.. por enquadramento e reinicia em cada simulação
Falha CRC	Descarte em L2 e nenhuma camada superior do receptor é acionada
Eficiência global	Dados úteis gerados / octetos transmitidos e corresponde à fórmula do projeto
Eficiência dos quadros	Soma dos bytes de aplicação presentes em todos os quadros / octetos transmitidos e conta os dados novamente em cada salto
Controle nos quadros	Soma de H5, H4, H3, H2 e F2 efetivamente transportados, sem repetir H5 em todos os fragmentos
Falhas	Dados entregues são apresentados separadamente; não há retransmissão nem ACK na conta
Fora do modelo	Caso ARP real, preâmbulo, intervalo entre quadros, padding Ethernet, tempos físicos e protocolos completos
Desempate entre rotas	Os caminhos candidatos são ordenados pelo custo. Em caso de empate, utiliza-se a sequência de nomes dos roteadores, tornando a escolha determinística.

A tabela seguinte apresenta os resultados da topologia de referência, mantendo as configurações originais de cada cenário. Alterações na mensagem, na topologia, nas falhas ou no limite de segmentação podem produzir outros valores.

Caso	Úteis	Transmitidos	Quadros	Eficiência global	Entregas
C1	42 B	92 B	1	45,65%	1
C2	42 B	368 B	4	11,41%	1
C3	84 B	736 B	8	11,41%	2
C4	42 B	368 B	4	11,41%	1
C5	42 B	92 B	1	45,65%	0
C6	42 B	276 B	3	15,22%	0
C7	100 B	968 B	12	10,33%	1

Em E1, uma mensagem de 42 B atravessa um único enlace, resultando em eficiência global de $42/92 = 45,65\%$. Em E2, a mesma quantidade de dados atravessa quatro enlaces, resultando em $42/368 = 11,41\%$.

As eficiências relacionam dados e bytes transmitidos e não representam a taxa de sucesso da entrega. Por isso, C5 e C6 apresentam eficiência calculada, embora nenhuma mensagem chegue à aplicação.

9. Dependências e ambiente

O projeto foi desenvolvido em Python, utilizando primordialmente os recursos da biblioteca padrão para toda a execução da aplicação e lógica do simulador. Além disso, foram empregadas as bibliotecas ReportLab e Pillow para o suporte à geração editorial e manipulação de documentos e imagens. Para a distribuição no ambiente Windows, o executável final foi gerado por meio do PyInstaller, garantindo o empacotamento completo dos módulos e dependências do sistema.

O simulador utiliza somente módulos da biblioteca padrão do Python para sua lógica e para o servidor HTTP local. A interface é desenvolvida em HTML, CSS e JavaScript e requer um navegador com JavaScript habilitado.

O arquivo SimuladorOSI.exe incorpora o interpretador Python, os módulos necessários e os arquivos da interface. Dessa forma, sua utilização não exige instalação de Python ou de bibliotecas adicionais. O arquivo topologia.json permanece externo, na mesma pasta do executável, permitindo substituir a rede simulada sem gerar outro executável. A comunicação entre o navegador e o programa ocorre no próprio computador, sem necessidade de acesso à internet.